

P0571R2

Type Requirements for `<numeric>` Algorithms

Published Proposal, 2018-10-08

Author:

[Bryce Adelstein Lelbach](#) (NVIDIA)

This Version:

<https://wg21.link/P0571R2>

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LWG, LEWG, SG1

Table of Contents

1	Overview
1.1	Kinds of Algorithms in <code><numeric></code>
1.2	In-Order-Accumulator Algorithms
1.3	GENERALIZED_NONCOMMUTATIVE_SUM and GENERALIZED_SUM Algorithms
2	What Should the Intermediate Type Be?
2.1	Intermediate Type for Ordered <code><numeric></code> Algorithms with Initial Value Parameters
2.2	Intermediate Type for Ordered <code><numeric></code> Algorithms without Initial Value Parameters
2.3	Intermediate Type for Unordered <code><numeric></code> Algorithms
2.4	Intermediate Type Policy for <code><numeric></code> Algorithms
3	Type Requirements
4	Wording
4.1	Questions for LWG
	References
	Informative References

§ 1. Overview

For over a decade, the C++ standard library algorithms in the `<numeric>` header have had insufficient or unclear type requirements. This paper identifies some of the issues and proposes potential solutions.

[This chart](#) provides an overview of the current state of the type requirements of the `<numeric>` algorithms.

This proposal is not intended to change the existing design, specify any previous unspecified behavior which major implementations do not already conform to, or remove functionality. It simply clarifies and improves the specification of the `<numeric>` algorithms.

§ 1.1. Kinds of Algorithms in `<numeric>`

There are three kinds of algorithms in `<numeric>`:

- `accumulate`, `inner_product`, `partial_sum` and `adjacent_difference` have "in-order-accumulator" style wording.
- `exclusive_scan`, `inclusive_scan`, `transform_exclusive_scan`, `transform_inclusive_scan` are specified using `GENERALIZED_NONCOMMUTATIVE_SUM`.
- `reduce`, `transform_reduce` are specified via `GENERALIZED_SUM`.

1.2. In-Order-Accumulator Algorithms

The wording for all of these algorithms fits the following pattern:

- Create an `acc` object which is initialized with `init` if the algorithm signature contains an initial value parameter and `*first` otherwise.
- For (the) iterator(s) in the range(s) **in order**, modify `acc` by applying a binary update operation which takes `acc` and the dereferenced value(s) of the iterator(s) as arguments: e.g. `acc = binary_op(acc, *i)`.

The ordering requirement is necessary to ensure that these algorithms are well-defined for **non-associative** and/or **non-commutative** operations, such as floating-point addition and multiplication (non-associative and commutative), and subtraction (non-associative and non-commutative).

Currently, the wording forces each iteration to depend on the prior iteration to ensure the correct ordering of accumulation. This introduces a loop-carried dependency that makes it impossible to parallelize.

1.3. GENERALIZED_NONCOMMUTATIVE_SUM and GENERALIZED_SUM Algorithms

To parallelize these operations, we need to be able to re-order applications of the operations, partition the workload into sub-tasks and then combine the results of the sub-tasks together using the operator.

This, however, would give a nondeterministic result for non-associative or non-commutative operations; for example, floating point arithmetic.

In addition to adding entirely new `<numeric>` algorithms, the Parallelism TS introduced new algorithms which perform the same operations as `accumulate`, `inner_product` and `partial_sum`, but have weaker constraints that allow parallelization:

- `reduce` is a parallelizable variant of `accumulate` and `transform_reduce` is a parallelizable variant `inner_product`. They may produce nondeterministic results for non-associative or non-commutative operations.
- `inclusive_scan` is a parallelizable variant of `partial_sum`. It may produce nondeterministic results for non-associative operations, but is fine for non-commutative operations.

These `<numeric>` algorithms are specified in [\[numerics.defns\]](#) using `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`:

24.2 Definitions

[numerics.defns]

1 Define `GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aN)` as follows:

(1.1) • `a1` when `N` is 1, otherwise either

(1.2) • `op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK),
GENERALIZED_NONCOMMUTATIVE_SUM for any K where $1 < K + 1 = M \leq N$, or`

2 Define `GENERALIZED_SUM(op, b1, ..., bN)` where `b1, ..., bN` may be any permutation of `a1, ..., aN`.

This definition allows:

- Arbitrary, nested partitioning of input elements for both `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`.
- Arbitrary reordering of input elements for `GENERALIZED_SUM`.

§ 2. What Should the Intermediate Type Be?

During computation of the final result, a `<numeric>` algorithm needs to store the result of accumulation thus far in temporary objects. The **intermediate type** is the type of these objects. Importantly, these temporary objects are passed as the first argument to the binary operator. For the accumulator-style `<numeric>` algorithms (`accumulate`, `inner_product`, `partial_sum`, `adjacent_difference`), the intermediate type is the type of the accumulator object `acc`.

The intermediate type is only clearly specified for 2 of the 10 `<numeric>` algorithms. Determining and specifying the intermediate type for these algorithms is our first step because the type requirements all revolve around the intermediate type.

§ 2.1. Intermediate Type for Ordered `<numeric>` Algorithms with Initial Value Parameters

`accumulate` and `inner_product` are described by the standard in [\[accumulate\]](#) and [\[inner.product\]](#). Today, the definitions don't make it clear what the intermediate type is. Both algorithms have a requirement that the type of the initial value parameter (`T`) be `CopyConstructible` and `CopyAssignable`, implying that the accumulator object is intended to be of type `T`. Both `libc++` and `libstdc++` use `T` as the type of the accumulator object for these 2 algorithms.

Using `T` both has upsides:

```
vector<int> i{INT_MAX, INT_MAX};
big_int bi = accumulate(d.begin(), d.end(), big_int(0));
// big_int is an arbitrary-precision integer class which uses dynamic storage.

// bi == 2 * INT_MAX.
```

and downsides:

```
vector<double> d{0.5, 0.5};
double r = accumulate(d.begin(), d.end(), 0);

// r == 0, not 1. The accumulator's type was int, since the type of the literal
// 0 is int.
```

Alternative choices for the intermediate type are:

- Determine some common type between `T`, `InputIterator`'s value type, and the result of `binary_op` (or the relevant operator) and use that as the intermediate type. As discussed in [§2.2 Intermediate Type for Ordered `<numeric>` Algorithms without Initial Value Parameters](#), it may be difficult to do this if `binary_op` is overloaded in a certain manner and it may make it harder for users to determine what the intermediate type will be.
- Use the `InputIterator`'s value type as the intermediate type. This would make the example of accumulating a `vector<double>` produce a result that might be closer to what the user intended, but it would also remove the functionality desired by the user in the `big_int` example.

Switching to either of these alternatives would force implementations to make breaking changes, and neither option seems particularly attractive.

I suggest adopting the following design. It is simple and clear to both users and implementors:

`<numeric>` algorithms which take an initial value parameter and initial value type template parameter will use the initial value type as the intermediate type.

§ 2.2. Intermediate Type for Ordered <numeric> Algorithms without Initial Value Parameters

As we mentioned earlier, the type of the accumulator object (the intermediate type) is explicitly specified for only 2 of the 10 <numeric> algorithms: `partial_sum`, which is specified in [\[partial.sum\]](#), and `adjacent_difference`, which is specified in [\[adjacent.difference\]](#)..

The only alternative to using `InputIterator`'s value type for the intermediate type that I could think of was computing the common type of the `InputIterator`'s value type and the result of the right-hand side of the repeated assignment to `acc`, e.g.

```
using it_value_type = typename iterator_traits<InputIterator>::value_type;

// The accumulator type, determined via common_type.
using acc_common_type = common_type_t<
    it_value_type
    , decltype(binary_op(it_value_type{}, *i))
    // Or acc + *i, or acc - *i, etc.
>
```

If the `InputIterator`'s value type is convertible to the result of the binary operator, but the result of the binary operator is not convertible to the `InputIterator`'s value type, then the binary operator signature we tested with `decltype` cannot be called with `acc_common_type` as its first argument:

```
struct A { };
struct B { operator A(); };

struct binary_op_type
{
    A operator() (B, B);
};

binary_op_type binary_op;

using it_value_type = B;

using acc_common_type = common_type_t<
    it_value_type
    , decltype(binary_op(it_value_type{}, it_value_type{}))
>;

int main()
{
    binary_op(acc_common_type{}, it_value_type{}); // COMPILE FAIL.
}
```

Even worse, we could have a `binary_op` like this:

```
struct binary_op_type
{
    A operator() (B, B);
    int operator() (A, B);
};
```

The above `binary_op` can now be called with `acc_common_type` as the first argument, however that overload returns a different type which we did not include in our common type computation. Nor could we have, as an iterative TMP search for a common type would be dangerous in the face of potential cycles:

```
struct binary_op_type
{
    A operator() (B, B); // New expression binary_op(A{}, B{}) to check...
    B operator() (A, B); // New expression binary_op(B{}, B{}) to check...
};
```

This might be viable if we constrain `binary_op` in some fashion, but it is not clear to me how that could be done. More importantly, determining a common type to use for the intermediate type is likely the

wrong thing to do because it means the user does not have a clear answer to the question "What type will you use to perform the accumulation?".

Since the `OutputIterator` has no value type by virtue of being an output iterator, I cannot think of any other options for the intermediate type for the algorithms without initial value parameters other than the status quo of the `InputIterator`'s value type.

There is, however, a regrettable lack of functionality with the status quo prior to C++17:

```
vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
partial_sum(f.begin(), f.end(), back_inserter(d));
// d[1] == inf since the intermediate type is float (the value type of
// vector<float>::iterator).
```

Pre C++17, there is no way for users to specify that `partial_sum` should use a particular type as the intermediate type instead of the `InputIterator`'s value type. In C++17, there are two ways this can be done if ordered is not needed. `inclusive_scan`, the unordered counterpart of `partial_sum`, has overloads which take an initial value:

```
vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
inclusive_scan(f.begin(), f.end(), back_inserter(d), plus<>{}, double{0.0});
// d[1] == 2 * FLT_MAX
```

or with `transform_inclusive_scan` using a transform function that converts its argument and returns the desired type:

```
vector<float> f{FLT_MAX, FLT_MAX};
vector<double> d;
transform_inclusive_scan(f.begin(), f.end(), back_inserter(d)
                        , plus<>{}
                        , [](float f) { return double{f}; });
// d[1] == 6e38F.
```

A possible post-C++17 addition of `partial_sum` signatures accepting an initial value parameter would complete this functionality.

I believe the best option is to keep the existing behavior for `<numeric>` algorithm which do not take an initial value parameter. Making any change to the type of the accumulator object would be a breaking change as `partial_sum` and `adjacent_difference` are currently specified to use accumulator objects whose type is the `InputIterator`'s value type, and none of the alternatives offer much benefit. The current behavior is clear and easy to understand.

So, I propose:

<numeric> algorithms which do not take an initial value parameter and initial value type template parameter will use the value type of their input iterator type as the intermediate type.

§ 2.3. Intermediate Type for Unordered `<numeric>` Algorithms

Unlike the ordered algorithms, the new unordered algorithms from the Parallelism TS v1 do not use in-order-accumulator wording. Instead, they are all specified in terms of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` ([§1.3 GENERALIZED_NONCOMMUTATIVE_SUM and GENERALIZED_SUM Algorithms](#)).

As an example of how these definitions are used, let's take a look at `reduce`, which is defined in [\[reduce\]](#). This definition does not clearly state what the intermediate type (e.g. the return type of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`) should be.

This is particularly scary in the face of the arbitrary reordering and partitioning that `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` allow. Consider:

```

vector<int> i{INT_MAX, INT_MAX, INT_MAX, INT_MAX};
big_int bi = reduce(d.begin(), d.end(), big_int(0));

// Possible result
// GSUM == GENERALIZED_SUM, GNSUM == GENERALIZED_NONCOMMUTATIVE_SUM
//
// bi = GSUM(operator+, big_int(0), d[0], d[1], d[2], d[3]);
//      = operator+(GNSUM(operator+, d[0], big_int(0))
//                  , GNSUM(operator+, d[1], d[2], d[3]));
//      = operator+(
//          operator+(
//              GNSUM(operator+, d[0]), GNSUM(operator+, big_int(0))
//          )
//      , operator+(
//          GNSUM(operator+, d[1], d[2]), GNSUM(operator+, d[3])
//      )
//      );
//      = operator+(
//          operator+(d[0], big_int(0))
//      , operator+(
//          operator+(GNSUM(operator+, d[1]), GNSUM(operator+, d[2])), d[3]
//      )
//      );
//      = operator+(
//          operator+(d[0], big_int(0))
//      , operator+(operator+(d[1], d[2]), d[3])
//      );
//      = ((d[0] + big_int(0)) + ((d[1] + d[2]) + d[3]));
//      = ((d[0] + big_int(0)) + ((int{INT_MAX} + int{INT_MAX}) + d[3]));
//      = ((d[0] + big_int(0)) + (int{-2} + int{INT_MAX}));
//      = ((int{INT_MAX} + big_int(0)) + int{INT_MAX - 2});
//      = (big_int(INT_MAX) + int{INT_MAX - 2});
//      = big_int(INT_MAX + INT_MAX - 2);
//
// bi = 2 * INT_MAX - 2; // Instead of 4*INT_MAX

```

The above is just one possible result that `reduce` could produce in such an example. Note that in addition to performing some of the calculations with `int` as the intermediate type, `reduce` also called `binary_op` with the initial value type as the **second** argument and the element from the sequence (whose type is the `InputIterators`'s value type) as the **first** argument. The ordered algorithms always pass intermediate objects as the first argument to the binary operator. It seems sensible that we should use the same rules I suggested for the ordered algorithms ([§2.4 Intermediate Type Policy for <numeric> Algorithms](#)). Note that the `N == 1` case intentionally does not convert to `IntermediateType`,

The question is how should this be done? The first thing I tried was to simply add a type parameter to `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` and specify that the type parameter is the return type. However, because of the recursive definition, this approach would force every input element to be converted to `IntermediateType`. Revisiting our example of reducing into a `big_int` illustrates the problem:

```

vector<int> i(N, INT_MAX); // N is divisible by 2.
big_int bi = reduce(par, d.begin(), d.end(), big_int(0));
// big_int is an arbitrary-precision integer class which uses dynamic storage.

```

Suppose we execute this code on a system with two cores and our implementation decides to use two threads. We would want the following to happen:

0. Partition the work into two chunks - say `init` (which is `big_int(0)`) and `d[0]` through `d[N / 2 - 1]` in the first chunk, and `d[N / 2]` through `d[N - 1]` in the second trunk.
1. On one thread, sequentially accumulate the first chunk into `init` (passed by value, so we can use it), using the accumulator-in-order semantics. E.g. modify `init` with `init = binary_op(acc, d[i])` for every `i` in `[0, N / 2)` in order.
2. On the second thread, create an accumulator object `acc` whose type is the the type of the initial value parameter (`T`) and initialize it with `d[N / 2]`. Then, modify `acc` with `acc = binary_op(acc, d[i])` for

every i in $[N / 2 + 1, N)$ in order. Note that we exclude $N / 2$ since we initialized `acc` with the value of `d[N / 2]`.

With the definition of `reduce` in C++17, we could arguably get this behavior. Although the current definition of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` are fully recursive down to a single element, the intention was to allow recursion to be replaced with sequential, in-order-accumulator style semantics at an implementation's discretion for chunks of work that will be done on a single thread, the seq execution policy, and inputs that an implementation decides not to parallelize. Under the current wording, it certainly seems like this would be an allowable as-if optimization.

But the new definition I suggested would require every element to be converted to `big_int`, which we have noted is an arbitrary precision type which uses dynamic storage. These conversions are undesirable, and arguably, unnecessary. We only want to ensure that any time the implementation uses a temporary object to accumulate some part of the result, that object's type is the desired intermediate type and a conversion happens if necessary (e.g. if the object is not the `init` parameter).

I don't believe we can specify that behavior without changing the definitions of `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` to explicitly allow the sequential accumulation of sub-partitions at the implementation's discretion.

§ 2.4. Intermediate Type Policy for `<numeric>` Algorithms

In summary:

The intermediate type is the initial value type if there is an initial value, and the value type of the `InputIterator` otherwise.

§ 3. Type Requirements

Now that we have defined a clear policy for what the intermediate type should be for each `<numeric>` algorithm ([§2.4 Intermediate Type Policy for `<numeric>` Algorithms](#)), we can describe their requirements:

- Operators shall neither modify elements nor invalidate iterators or subranges in the iterator ranges specified by their parameters.
- The intermediate type shall be `CopyConstructible` (in-order-accumulator algorithms that take an initial value), constructible from `*first` (in-order-accumulator algorithms that do not take an initial value), or constructible from `op(GENERALIZED_NONCOMMUTATIVE_SUM(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM(op, aM, ..., aN))` (`GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` algorithms). This requirement comes from the initialization of the accumulator object (in-order accumulator algorithms) or from the conversion to `IntermediateType` (`GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM` algorithms).
- The right hand side of the update expression must be convertible to the intermediate type (all in-order-accumulator algorithms). This requirement comes from the update expression (the right hand side of `acc = binary_op(acc, *i)`).
- The intermediate type must be `CopyAssignable` (all in-order-accumulator algorithms except `adjacent_difference`) or `MoveAssignable` (`adjacent_difference`). This requirement comes from the update expression (the assignment in `acc = binary_op(acc, *i)`).
- The intermediate type shall be writable to the result output iterator (for algorithms which take an output range). This requirement comes from the write to the output iterator that is performed each iteration in these algorithms.

For the function objects, we've already specified the requirements for what the return type of the invocations of `binary_op` should be. We just need to require that:

- `binary_op`, `binary_op1`, `binary_op2` and `unary_op` are function objects.

- `binary_op(acc, *i)` (all in-order-accumulator algorithms), `acc + *i` (accumulate, inner_product and partial_sum), `acc - *i` (adjacent_difference), `binary_op1(acc, binary_op2(*i1, *i2))` (inner_product) and `acc + (*i1) * (*i2)` are required.
- The function object requirements from the new algorithms are in `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`:
 - If `a1, ..., aN` are of the same type (e.g. the homogeneous elements of an iterator range), then `op(a1, a1)` is the only signature required by `GENERALIZED_NONCOMMUTATIVE_SUM` and `GENERALIZED_SUM`. This covers the new algorithms which do not take an initial value type.
 - If `a1` is of one type and `a2, ..., aN` are of the same type (e.g. an initial value `a1` and the homogeneous elements of an iterator range `a2, ..., aN`), then `op(a2, a1)`, `op(a1, a2)`, `op(a2, a2)` and `op(a1, a1)` are required by `GENERALIZED_SUM` and `op(a2, a1)`, `op(a2, a2)` and `op(a1, a1)` are required for `GENERALIZED_NONCOMMUTATIVE_SUM`. This covers the new algorithms which do take an initial value type.

While the new algorithms introduced by the Parallelism TS do have the necessary requirements forbidding modification of iterator ranges, the ranges used are not fully closed like the rest of the `<numeric>` algorithms.

There is one additional outlier case. `reduce` has a signature which only takes two `InputIterator`'s and is defined as equivalent to `return reduce(first, last, typename iterator_traits<InputIterator>::value_type{});`. This requires that `InputIterator`'s value type be `DefaultConstructible`. `accumulate` does not have such a signature (by design, I believe). Ideally, I'd like to remove this signature, but I think it is too late to do so. I'd like to add the missing requirement, at least.

§ 4. Wording

Note: The following changes are relative to the post-Rapperswil 2018 working draft of ISO/IEC 14882, ([\[N4762\]](#)).

Note: The $\blacklozenge$ character is used to denote a placeholder number which shall be selected by the editor.

Modify 24.2 [\[numerics.defns\]](#) as follows:

- ❖ Let `IntermediateType` be a type. Let $a_1, \dots, a_N$ and $b_1, \dots, b_N$ be expressions whose types are implicitly convertible to `IntermediateType`.
 - ❖ `IntermediateType` shall be `Cpp17MoveConstructible` (Table 25), `Cpp17MoveAssignable` (Table 27), and constructible from a_1 .
 - ❖ Let `op` is a binary function object (19.14 [function.object]) taking two arguments, the first whose type is `IntermediateType` and second whose type is implicitly convertible to `IntermediateType`, and returns a value whose type is implicitly convertible to `IntermediateType`.
- 1 Define the expression `GENERALIZED_NONCOMMUTATIVE_SUM<IntermediateType>(op, a1, ..., aN)` as follows:
 - (1.1) • `IntermediateType(a1)` when N is 1, otherwise either
 - (1.2) • `IntermediateType(op(GENERALIZED_NONCOMMUTATIVE_SUM<IntermediateType>(op, a1, ..., aK), GENERALIZED_NONCOMMUTATIVE_SUM<IntermediateType>(op, aM, ..., aN)))` for any K where $1 < K + 1 = M \leq N$, or
 - (1.3) • initialize the accumulator `acc` whose type is `IntermediateType` with the value `a1`, modify it with `acc = op(move(acc), GENERALIZED_NONCOMMUTATIVE_SUM<IntermediateType>(op, aJ, ..., aK))` for every $[J, K]$ in $[2, X), \dots, [Y, N]$ where $2 < X + 1 < \dots < Y + 1 \leq N$ in order, and then return `acc`.
 - 2 Define `GENERALIZED_SUM<IntermediateType>(op, a1, ..., aN)` as `GENERALIZED_NONCOMMUTATIVE_SUM<IntermediateType>(op, b1, ..., bN)` where $b_1, \dots, b_N$ may be any permutation of $a_1, \dots, a_N$.

Modify 23.9.1 [accumulate] as follows:

23.9.1 Accumulate

[accumulate]

```
template<class InputIterator, class T>
    T accumulate(InputIterator first, InputIterator last, T init);
template <class InputIterator, class T, class BinaryOperation>
    T accumulate(InputIterator first, InputIterator last, T init,
                 BinaryOperation binary_op);
```

- 1 *Requires:* T shall satisfy the `Cpp17MoveCopyConstructible` (Table 2526) and `Cpp17MoveCopyAssignable` (Table 2728) requirements. The result of the expression `acc + *i` or `binary_op(std::move(acc), *i)` shall be implicitly convertible to T . In the range $[first, last]$, `binary_op` shall neither modify elements nor invalidate iterators or subranges.²⁴⁰
- 2 *Effects:* Computes its result by initializing the accumulator `acc`, whose type is T , with the initial value `move(init)` and then modifies it with `acc = std::move(acc) + *i` or `acc = binary_op(std::move(acc), *i)` for every iterator i in the range $[first, last)$ in order.²⁴¹

²⁴⁰⁾ The use of fully closed ranges is intentional.

²⁴¹⁾ `accumulate` is similar to the APL reduction operator and Common Lisp `reduce` function, but it avoids the difficulty of defining the result of reduction on an empty sequence by always requiring an initial value.

Modify 23.9.2 [reduce] as follows:

23.9.2 Reduce

[reduce]

```
template<class InputIterator>
```

```
    typename iterator_traits<InputIterator>::value_type  
    reduce(InputIterator first, InputIterator last);
```

1 *Effects:* Equivalent to:

```
    return reduce(first, last,  
                  typename iterator_traits<InputIterator>::value_type{});
```

◆ *Requires:* `iterator_traits<InputIterator>::value_type` shall satisfy the requirements of *Cpp17DefaultConstructible* (Table 20)

```
template<class ExecutionPolicy, class ForwardIterator>
```

```
    typename iterator_traits<ForwardIterator>::value_type  
    reduce(ExecutionPolicy&& exec,  
           ForwardIterator first, ForwardIterator last);
```

2 *Effects:* Equivalent to:

```
    return reduce(std::forward<ExecutionPolicy>(exec), first, last,  
                  typename iterator_traits<ForwardIterator>::value_type{});
```

◆ *Requires:* `iterator_traits<ForwardIterator>::value_type` shall satisfy the requirements of *Cpp17DefaultConstructible* (Table 20)

```
template<class InputIterator, class T>
```

```
    T reduce(InputIterator first, InputIterator last, T init);
```

3 *Effects:* Equivalent to:

```
    return reduce(first, last, init, plus<>());
```

```
template<class ExecutionPolicy, class ForwardIterator, class T>
```

```
    T reduce(ExecutionPolicy&& exec,  
           ForwardIterator first, ForwardIterator last, T init);
```

4 *Effects:* Equivalent to:

```
    return reduce(std::forward<ExecutionPolicy>(exec), first, last, init, plus<>());
```

```
template<class InputIterator, class T, class BinaryOperation>
```

```
    T reduce(InputIterator first, InputIterator last, T init,  
           BinaryOperation binary_op);
```

```
template<class ExecutionPolicy, class ForwardIterator, class T, class BinaryOperation>
```

```
    T reduce(ExecutionPolicy&& exec,  
           ForwardIterator first, ForwardIterator last, T init,  
           BinaryOperation binary_op);
```

5 *Requires:* `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last]`.

(5.1) • ~~`T` shall be *Cpp17MoveConstructible* (Table 25).~~

(5.2) • ~~All of `binary_op(init, *first)`, `binary_op(*first, init)`, `binary_op(init, init)`, and `binary_op(*first, *first)` shall be convertible to `T`.~~

(5.3) • ~~`binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last]`.~~

6 *Returns:* `GENERALIZED_SUM<T>(binary_op, init, *i, ...)` for every `i` in `[first, last]`.

7 *Complexity:* $O(\text{last} - \text{first})$ applications of `binary_op`.

8 [*Note:* The difference between `reduce` and `accumulate` is that `reduce` applies `binary_op` in an unspecified order, which yields a nondeterministic result for non-associative or non-commutative `binary_op` such as floating-point addition. – *end note*]

Modify 23.9.3 [\[inner.product\]](#) as follows:

23.9.3 Inner product

[inner.product]

```
template<class InputIterator1, class InputIterator2, class T>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, T init);
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, T init,
                    BinaryOperation1 binary_op1,
                    BinaryOperation2 binary_op2);
```

- 1 *Requires:* T shall satisfy the Cpp17 **MoveCopyConstructible** (Table 2526) and Cpp17 **MoveCopyAssignable** (Table 2728) requirements. The result of the expression `std::move(acc) + (*i1) * (*i2)` or `binary_op1(std::move(acc), binary_op2(*i1, *i2))` shall be implicitly convertible to T. In the ranges [first1, last1] and [first2, first2 + (last1 - first1)] binary_op1 and binary_op2 shall neither modify elements nor invalidate iterators or subranges.²⁴²
- 2 *Effects:* Computes its result by initializing the accumulator acc, whose type is T, with the initial value `move(init)` and then modifying it with `acc = std::move(acc) + (*i1) * (*i2)` or `acc = binary_op1(std::move(acc), binary_op2(*i1, *i2))` for every iterator i1 in the range [first1, last1) and iterator i2 in the range [first2, first2 + (last1 - first1)) in order.

²⁴²⁾ The use of fully closed ranges is intentional.

Modify 23.9.4 [transform.reduce] as follows:

23.9.4 Transform reduce

[transform.reduce]

```
template<class InputIterator1, class InputIterator2, class T>
    T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                      InputIterator2 first2,
                      T init);
```

- 1 *Effects:* Equivalent to:
`return transform_reduce(first1, last1, first2, init, plus<>(), multiplies<>());`

```
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T>
    T transform_reduce(ExecutionPolicy&& exec,
                      ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2,
                      T init);
```

- 2 *Effects:* Equivalent to:
`return transform_reduce(std::forward<ExecutionPolicy>(exec),
 first1, last1, first2, init, plus<>(), multiplies<>());`

```
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
    T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                      InputIterator2 first2,
                      T init,
                      BinaryOperation1 binary_op1,
                      BinaryOperation2 binary_op2);
```

```
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
    T transform_reduce(ExecutionPolicy&& exec,
                      ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2,
                      T init,
                      BinaryOperation1 binary_op1,
                      BinaryOperation2 binary_op2);
```

- 3 *Requires:* Neither binary_op1 nor binary_op2 shall invalidate subranges, or modify elements in the ranges [first1, last1] and [first2, first2 + (last1 - first1)].

- (3.1) • ~~T shall be Cpp17MoveConstructible (Table 25).~~
- (3.2) • All of
 - (3.1) ◦ ~~binary_op1(init, init);~~
 - (3.2) ◦ ~~binary_op1(init, binary_op2(*first1, *first2));~~
 - (3.3) ◦ ~~binary_op1(binary_op2(*first1, *first2), init), and~~
 - (3.4) ◦ ~~binary_op1(binary_op2(*first1, *first2), binary_op2(*first1, *first2));~~

~~shall be convertible to T.~~
- (3.3) • ~~Neither binary_op1 nor binary_op2 shall invalidate subranges, or modify elements in the ranges [first1, last1] and [first2, first2 + (last1 - first1)].~~

4 *Returns:*

GENERALIZED_SUM<T>(binary_op1, init, binary_op2(*i, *(first2 + (i - first1))), ...)

for every iterator i in [first1, last1).

5 *Complexity:* O(last1 - first1) applications each of binary_op1 and binary_op2.

```
template<class InputIterator, class T,
         class BinaryOperation, class UnaryOperation>
T transform_reduce(InputIterator first, InputIterator last, T init,
                  BinaryOperation binary_op, UnaryOperation unary_op);

template<class ExecutionPolicy,
         class ForwardIterator, class T,
         class BinaryOperation, class UnaryOperation>
T transform_reduce(ExecutionPolicy& exec,
                  ForwardIterator first, ForwardIterator last,
                  T init, BinaryOperation binary_op, UnaryOperation unary_op);
```

6 *Requires:* Neither unary_op nor binary_op shall invalidate subranges, or modify elements in the ranges [first1, last1].

- (6.1) • ~~T shall be Cpp17MoveConstructible (Table 25).~~
- (6.2) • All of
 - (6.1) ◦ ~~binary_op(init, init);~~
 - (6.2) ◦ ~~binary_op(init, unary_op(*first));~~
 - (6.3) ◦ ~~binary_op(unary_op(*first), init), and~~
 - (6.4) ◦ ~~binary_op(unary_op(*first), unary_op(*first1, *first2));~~

~~shall be convertible to T.~~
- (6.3) • ~~Neither unary_op nor binary_op shall invalidate subranges, or modify elements in the ranges [first1, last1].~~

7 *Returns:* GENERALIZED_SUM<T>(binary_op, init, unary_op(*i), ...) for every iterator i in [first, last).

8 *Complexity:* O(last - first) applications each of unary_op and binary_op.

9 [Note: transform_reduce does not apply unary_op to init. — end note]

Modify 23.9.5 [\[partial.sum\]](#) as follows:

23.9.5 Partial sum

[partial.sum]

```
template<class InputIterator, class OutputIterator>
    OutputIterator partial_sum(
        InputIterator first, InputIterator last,
        OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator partial_sum(
        InputIterator first, InputIterator last,
        OutputIterator result, BinaryOperation binary_op);
```

- 1 *Requires:* InputIterator's value type shall be [Cpp17CopyConstructible \(Table 26\)](#) and [Cpp17MoveAssignable \(Table 27\)](#) ~~constructible from the type of *first~~. The result of the expression `std::move(acc) + *i` or `binary_op(std::move(acc), *i)` shall be implicitly convertible to InputIterator's value type. `acc` shall be writable (22.2.1 [\[iterator.requirements.general\]](#)) to the result output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.²⁴³
- 2 *Effects:* For a non-empty range, the function creates an accumulator `acc` whose type is InputIterator's value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, `acc` is then modified by `acc = std::move(acc) + *i` or `acc = binary_op(std::move(acc), *i)` and the result is assigned to `*(result + (i - first))`.
- 3 *Returns:* `result + (last - first)`.
- 4 *Complexity:* Exactly `(last - first) - 1` applications of the binary operation.
- 5 *Remarks:* `result` may be equal to `first`.

²⁴³) The use of fully closed ranges is intentional.

Modify 23.9.6 [\[exclusive.scan\]](#) as follows:


```

2 Effects: Equivalent to:
   return inclusive_scan(std::forward<ExecutionPolicy>(exec), first, last, result, plus<>())
   );

template<class InputIterator, class OutputIterator, class BinaryOperation>
   OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                               OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
   ForwardIterator2 inclusive_scan(ExecutionPolicy&& exec,
                                   ForwardIterator1 first, ForwardIterator1 last,
                                   ForwardIterator2 result, BinaryOperation binary_op);

template<class InputIterator, class OutputIterator, class BinaryOperation, class T>
   OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                               OutputIterator result, BinaryOperation binary_op, T init);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class BinaryOperation, class T>
   ForwardIterator2 inclusive_scan(ExecutionPolicy&& exec,
                                   ForwardIterator1 first, ForwardIterator1 last,
                                   ForwardIterator2 result, BinaryOperation binary_op, T init
   );

```

◆ If `init` is not provided, `T` is the value type of `decltype(*first)`.

3 *Requires:*

- (3.1) • ~~If `init` is provided, `T` shall be *Cpp17MoveConstructible* (Table 25); otherwise, `ForwardIterator1`'s value type shall be *Cpp17MoveConstructible*.~~
- (3.2) • ~~If `init` is provided, all of `binary_op(init, init)`, `binary_op(init, *first)`, and `binary_op(*first, *first)` shall be convertible to `T`; otherwise, `binary_op(*first, *first)` shall be convertible to `ForwardIterator1`'s value type.~~
- (3.3) • `T` shall be writable (22.2.1 [\[iterator.requirements.general\]](#)) to the result output iterator.
- (3.4) • `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last]` or `[result, result + (last - first)]`.

4 *Effects:* For each integer `K` in `[0, last - first)` assigns through `result + K` the value of

- (4.1) `GENERALIZED_NONCOMMUTATIVE_SUM<T>(`
`binary_op, init, *(first + 0), *(first + 1), ..., *(first + K))`
 if `init` is provided, or
- (4.2) `GENERALIZED_NONCOMMUTATIVE_SUM<T>(`
`binary_op, *(first + 0), *(first + 1), ..., *(first + K))`
 otherwise.

5 *Returns:* The end of the resulting range beginning at `result`.

6 *Complexity:* $O(\text{last} - \text{first})$ applications of `binary_op`.

7 *Remarks:* `result` may be equal to `first`.

8 [*Note:* The difference between `exclusive_scan` and `inclusive_scan` is that `inclusive_scan` includes the i^{th} input element in the i^{th} sum. If `binary_op` is not mathematically associative, the behavior of `inclusive_scan` may be nondeterministic. — *end note*]

Modify 23.9.8 [\[transform.exclusive.scan\]](#) as follows:

23.9.8 Transform exclusive scan

[transform.exclusive.scan]

```
template<class InputIterator, class OutputIterator, class T,
        class BinaryOperation, class UnaryOperation>
OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result, T init,
                                       BinaryOperation binary_op, UnaryOperation unary_op)
;
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2 transform_exclusive_scan(ExecutionPolicy&& exec,
                                       ForwardIterator1 first, ForwardIterator1 last,
                                       ForwardIterator2 result, T init,
                                       BinaryOperation binary_op, UnaryOperation unary_
op);
```

1 Requires:

- (1.1) • ~~T shall be Cpp17MoveConstructible (Table 25):~~
- (1.2) • ~~All of~~
 - (1.1) ◦ ~~binary_op(init, init);~~
 - (1.2) ◦ ~~binary_op(init, unary_op(*first)); and~~
 - (1.3) ◦ ~~binary_op(unary_op(*first), unary_op(*first))~~~~shall be convertible to T:~~
- (1.3) • T shall be writable (22.2.1 [iterator.requirements.general]) to the result output iterator.
- (1.4) • Neither unary_op nor binary_op shall invalidate iterators or subranges, nor modify elements in the range [first, last] or [result, result + (last - first)].

2 Effects: For each integer K in [0, last - first) assigns through result + K the value of:

```
GENERALIZED_NONCOMMUTATIVE_SUM<T>(
    binary_op, init,
    unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K - 1)))
```

3 Returns: The end of the resulting range beginning at result.

4 Complexity: O(last - first) applications each of unary_op and binary_op.

5 Remarks: result may be equal to first.

- 6 [Note: The difference between transform_exclusive_scan and transform_inclusive_scan is that transform_exclusive_scan excludes the i^{th} input element from the i^{th} sum. If binary_op is not mathematically associative, the behavior of exclusive_scan may be nondeterministic. transform_exclusive_scan does not apply unary_op to init. — end note]

Modify 23.9.9 [transform.inclusive.scan] as follows:

23.9.9 Transform inclusive scan

[transform.inclusive.scan]

```

template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       BinaryOperation binary_op, UnaryOperation unary_op
);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                       ForwardIterator1 first, ForwardIterator1 last,
                                       ForwardIterator2 result,
                                       BinaryOperation binary_op, UnaryOperation unary_
op);
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation, class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       BinaryOperation binary_op, UnaryOperation unary_op
,
                                       T init);
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation, class T>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                       ForwardIterator1 first, ForwardIterator1 last,
                                       ForwardIterator2 result,
                                       BinaryOperation binary_op, UnaryOperation unary_
op,
                                       T init);

```

◆ If `init` is not provided, `T` is the value type of `decltype(*first)`.

1 Requires:

- (1.1) • If `init` is provided, `T` shall be *Cpp17MoveConstructible* (Table 25); otherwise, `ForwardIterator1`'s value type shall be *Cpp17MoveConstructible*.
- (1.2) • If `init` is provided, all of
 - (1.1) ◦ `binary_op(init, init)`;
 - (1.2) ◦ `binary_op(init, *first)`; and
 - (1.3) ◦ `binary_op(*first, *first)`
 shall be convertible to `T`; otherwise, `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to `ForwardIterator1`'s value type.
- (1.3) • `T` shall be *writable* (22.2.1 [iterator.requirements.general]) to the result output iterator.
- (1.4) • Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, nor modify elements in the range `[first, last)` or `[result, result + (last - first))`.

2 Effects: For each integer `K` in `[0, last - first)` assigns through `result + K` the value of

- (2.1) `GENERALIZED_NONCOMMUTATIVE_SUM<T>(`
 `binary_op, init,`
 `unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K))`
 `)`
 if `init` is provided, or
- (2.2) `GENERALIZED_NONCOMMUTATIVE_SUM<T>(`
 `binary_op,`
 `unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K))`
 `)`
 otherwise.

- 3 *Returns:* The end of the resulting range beginning at result.
- 4 *Complexity:* $O(\text{last} - \text{first})$ applications each of unary_op and binary_op.
- 5 *Remarks:* result may be equal to first.
- 6 [*Note:* The difference between transform_exclusive_scan and transform_inclusive_scan is that transform_inclusive_scan includes the i^{th} input element in the i^{th} sum. If binary_op is not mathematically associative, the behavior of inclusive_scan may be nondeterministic. transform_inclusive_scan does not apply unary_op to init. — *end note*]

Modify 23.9.10 [\[adjacent.difference\]](#) as follows:

```

template<class InputIterator, class OutputIterator>
    OutputIterator
        adjacent_difference(InputIterator first, InputIterator last, OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator2
        adjacent_difference(ExecutionPolicy&& exec,
                            ForwardIterator1 first, ForwardIterator1 last, ForwardIterator2 result,
                            BinaryOperation binary_op);

template<class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator
        adjacent_difference(InputIterator first, InputIterator last,
                            OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
    ForwardIterator2
        adjacent_difference(ExecutionPolicy&& exec,
                            ForwardIterator1 first, ForwardIterator1 last,
                            ForwardIterator2 result, BinaryOperation binary_op);

```

- 1 Let T be the value type of `decltype(*first)`. For the overloads that do not take an argument `binary_op`, let `binary_op` be an lvalue that denotes an object of type `minus<T>`.
- 2 *Requires:*
 - (2.1) • For the overloads with no `ExecutionPolicy`, T shall be *Cpp17MoveAssignable* (Table 27) and shall be constructible from the type of `*first`. `acc` (defined below) shall be writable (22.2.1 [\[iterator.requirements.general\]](#)) to the result output iterator.
 - (2.2) • The result of the expression `binary_op(val, std::move(acc))` shall be writable to the result output iterator.
 - (2.3) • For the overloads with an `ExecutionPolicy`, the result of the expressions `binary_op(*first, *first)` and `*first` shall be writable to result.
 - (2.4) • For all overloads, in the ranges `[first, last]` and `[result, result + (last - first)]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.²⁴⁴
 - (2.5) • For the overloads with an `ExecutionPolicy`, the ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.
- 3 *Effects:* For the overloads with no `ExecutionPolicy` and a non-empty range, the function creates an accumulator `acc` of type T , initializes it with `*first`, and assigns the result to `*result`. For every iterator i in `[first + 1, last)` in order, creates an object `val` whose type is T , initializes it with `*i`, computes `binary_op(val, std::move(acc))`, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`.
- 4 For the overloads with an `ExecutionPolicy` and a non-empty range, performs `*result = *first`. Then, for every d in `[1, last - first - 1]`, performs `*(result + d) = binary_op(*(first + d), *(first + (d - 1)))`.
- 5 *Returns:* `result + (last - first)`.
- 6 *Complexity:* Exactly `(last - first) - 1` applications of the binary operation.
- 7 *Remarks:* For the overloads with no `ExecutionPolicy`, result may be equal to first. ~~For the overloads with an ExecutionPolicy, the ranges `[first, last)` and `[result, result + (last - first))` shall not overlap.~~

²⁴⁴) The use of fully closed ranges is intentional.

- Do we `std::` qualify things in library wording, like `move` and `forward`?
- Do we prefer to say "Iterator's value type" or "`typename iterator_traits<Iterator>::value_type`".
- The "use of fully closed ranges is intentional" notes are used inconsistently - should they be everywhere?

§ References

§ Informative References

[N4762]

Richard Smith. [Working Draft, Standard for Programming Language C+](https://wg21.link/n4762). 7 July 2018. URL: <https://wg21.link/n4762>